

Amélioration du système de routage du simulateur d'Urbanloop grâce au paradigme SDN

Victor Thevenon

TELECOM Nancy - Promotion 2020

Email: victor.thevenon@telecomnancy.eu

Frédéric Venier

TELECOM Nancy - Promotion 2020

Email: frederic.venier@telecomnancy.eu

Thibault Cholez

Chercheur à l'INRIA

Enseignant à TELECOM Nancy

Email: thibault.cholez@inria.fr

Abstract—Ce document retrace un travail effectué sur l'amélioration d'un système de routage appliqué au moyen de transport urbain l'Urbanloop, en particulier s'adapter au flux de navettes ou aux problèmes ponctuels qui pourraient être rencontrés dans le réseau de navettes. Le but de cette étude est d'améliorer un simulateur déjà existant pour qu'il intègre les principes du paradigme SDN (Software Defined Networking) afin d'avoir un système de routage pouvant s'adapter dynamiquement, et de vérifier que cela améliore l'état global du réseau. L'article présente un état de l'art sur SDN et nos solutions pour l'implémenter au simulateur existant ainsi qu'une analyse des résultats obtenus. Il comporte également les résultats du travail effectué en particulier sur les défaillances du réseau. Nous n'avons cependant pas pu obtenir de résultats sur la gestion de congestions dans le réseau, bien que l'algorithme ait été implémenté, car le simulateur ne le permettait pas.

Index Terms—Urbanloop, transports intelligents, Software Defined Networking, algorithme de routage, simulation à événements discrets

I. INTRODUCTION

Le système de routage des capsules est un des piliers du projet Urbanloop. En effet c'est ce système qui va permettre de diriger les différentes capsules à travers le réseau de routes disponibles. Un simulateur a déjà été créé pour représenter le déplacement des capsules dans le réseau, cependant ce simulateur ne permet pas d'adapter dynamiquement le routage. En effet le but est d'avoir un réseau de capsules pouvant s'adapter à tout instant selon l'état d'un réseau, c'est à dire que dans le cas d'une route coupée, les capsules doivent changer de voie pour atteindre leur destination si cela est toujours possible. Également, dans le cas d'une forte affluence, les capsules doivent être routées dans l'optique d'éviter le plus possible la création de congestion dans le réseau et donc avoir un trafic plus fluide. Pour ce faire, il a été décidé d'appliquer le paradigme SDN (Software Defined Software) au réseau, celui-ci permettant d'avoir à tout moment le contrôle sur l'ensemble du réseau via un contrôleur. Des recherches ont donc été effectuées sur ce paradigme qui a ensuite été appliqué au simulateur existant. Cependant il n'a pas été possible, par manque de temps, d'adapter le simulateur fourni pour créer de la congestion, celui-ci n'ayant pas été conçu pour cela, mais nous avons quand même implémenté le code permettant de détecter et de s'adapter un cas de congestion du réseau.

Nous présenterons d'abord le paradigme SDN puis nous verrons comment nous l'avons implémenté sur le simulateur

d'Urbanloop. Nous terminerons par les résultats issus de notre travail.

II. ÉTUDE DE L'ARCHITECTURE SDN

A. Pourquoi SDN ?

Le paradigme SDN provient d'abord d'une préoccupation universitaire et a pris énormément d'importance depuis une dizaine d'années. Il est désormais en phase d'industrialisation et de grands groupes comme Google se sont emparés de la question. L'apparition de SDN vient en réponse à la complexité des réseaux lorsqu'ils sont utilisés avec le protocole IP, ce dernier posant un problème majeur de flexibilité.[1]

En effet ce dernier s'adapte peu aux changements. Par exemple le passage de l'IPv4 à l'IPv6 [2] (qui amènerait une plus grande flexibilité dans l'attribution des adresses et un routage plus facile) est largement ralenti par la configuration actuelle. Un autre exemple serait l'installation d'un pare feu sur un réseau multi-site qui nécessiterait de reconfigurer le réseau. Le problème du protocole IP est lié à son intégration verticale, un réseau s'organisant de la sorte[1] :

- Les gestionnaires réseau (General Services) : les développeurs, agents de maintenance, etc.
- Le plan de management (Management Plane) : les logiciels qui permettent la gestion du réseau.
- Le plan de contrôle (Control Plane): les protocoles qui définissent le routage dans le réseau.
- Le plan de données (ou plan de commutation, Data Plane) : les routeurs, switches, etc.

B. Le découplage du plan de données et du plan de contrôle

L'idée majeure du paradigme SDN est de découpler le plan de données et le plan de contrôle. Cette séparation vient répondre au problème posé par l'intégration verticale. Dans un réseau IP traditionnel le plan de contrôle qui décide de la route pour acheminer les données et le plan de données qui sert à les acheminer sont liés dans les routeurs. Il en résulte de nombreux problèmes et en particulier un manque de flexibilité. En effet ce système empêche la reconfiguration automatique du réseau et les modifications doivent souvent être effectuées pour chaque dispositif du réseau. De plus cela rend le réseau très dépendant des systèmes physiques.[1] Pour arriver à séparer le plan de données du plan de contrôle il faut que le contrôleur puisse s'adresser directement aux plans de données en utilisant une API (ex : protocole OpenFlow)[1][3]

C. Les quatre piliers de SDN

SDN s'appuie sur quatre principes fondamentaux :

- 1/ Découpler plan de contrôle (ou RIB) et plan de données (ou FIB).
- 2/ Un système de transmission flow-based (défini par des flux, qui sont des ensembles de paquets de données entre la source et la destination) plutôt que destination-based (on connaît seulement la destination, et non la source). Le commutateur va créer un "sink tree" pour acheminer la donnée, autrement dit il va calculer les chemins pour la destination indiquée en partant de toutes les sources.
- 3/ Le contrôle est exercé par une entité externe aux systèmes physiques appelée Network Operating System (NOS).
- 4/ Le réseau est contrôlable par les développeurs à travers des programmes en langage de haut-niveaux qui agissent sur le NOS.

D. Un réseau modifiable

SDN présente un énorme avantage : le réseau pourrait être modifié (modification des règles du réseau) à travers des logiciels et des langages de haut-niveau et cela de manière quasi-immédiate à travers le plan de management en intervenant sur le NOS. Comme le plan de contrôle serait centralisé dans le NOS, agir sur ce dernier permettrait de modifier tous les routeurs en même temps. Les routeurs ne serviraient plus que pour transmettre les paquets de données. Une couche spéciale du NOS serait responsable de l'installation des commandes sur les systèmes de commutation et de la récupération des informations. Une API (Southbound Interface) permettrait de communiquer entre le NOS et les routeurs[1].

E. Architecture générale

- Forwarding Devices (FD) : hardware ou software. Leur but est de transmettre les paquets. Ils ne connaissent que les flow-rules que leur transmet le NOS.
- Data Plane (DP) : l'ensemble des FD
- Control Plane (CP) : plan qui programme les Forwarding Devices (FD)
- Southbound Interface (SI) : API qui définit le protocole entre le Data Plane (DP) et le CP
- Northbound Interface (NI) : API entre le control plane et les développeurs
- Management Plane (MP) : les applications qui permettent de contrôler le réseau et notamment de créer les flow-rules

F. Fonctionnement

Le fonctionnement de SDN entraîne un besoin vis à vis des routeurs utilisés. Ceux ci devront en effet avoir une interface standardisée et ouverte de sorte que les couches supérieures puissent communiquer avec eux.[1][1].

Le routage SDN fonctionne sur une méthode de flow. C'est à dire que la transmission des données est assurée par un flux de paquets. Le routeur possède une flow table qui possède trois éléments[1][4][5] :

- Des règles d'attribution de paquets : Celles-ci sont transmises par les couches supérieures. Ces règles sont définies selon les différentes données portées par les paquets (adresse mac source, port switch, adresse IP, destination, etc.).
- Des instructions à effectuer pour chaque règle. Si une règle s'applique à un paquet, les instructions correspondantes seront effectuées.
- Des statistiques sur la transmission qui seront envoyées au plan de contrôle.

Chaque fois qu'un paquet arrive sur un routeur, on teste sa correspondance avec une règle. Si il ne correspond à aucune règle des flow tables il est alors détruit. Si il correspond à une règle, le routeur applique les instructions correspondantes. Ces instructions peuvent être[1] :

- Acheminer le paquet jusqu'au port sortant.
- Encapsuler et contacter le contrôleur.
- Lui faire suivre la procédure normale.
- Le transmettre jusqu'à la prochaine table.

G. Southbound Interface

Cette interface est primordiale dans le système car elle fait le lien entre le plan de contrôle et le plan de données. C'est donc elle qui fait le lien entre le contrôleur et les éléments de commutation. A travers cette API le plan de contrôle peut demander aux routeurs d'enclencher un lien, de changer de port, de transmettre des statistiques ou encore d'acheminer un flux[1].

OpenFlow est l'interface la plus courante. Des alternatives existent comme OVSD, POF, OpenState ou encore OpFlex.

H. Sécurité et SDN

Le paradigme SDN entraîne un risque sécuritaire important. En effet, la centralisation du contrôle change radicalement la sécurité du réseau par rapport à un réseau traditionnel.

Un individu qui aurait accès au réseau d'administration (management plane) peut directement modifier le fonctionnement du réseau. Ainsi il est nécessaire d'authentifier (Transport Layer Security (TLS)) les transmissions entre le plan de management et le plan de contrôle (Northbound API) et aussi de crypter les transmissions au niveau SouthBound et Northbound car celles-ci donnent des informations directes sur le réseau. [6]

I. Conclusion

Le paradigme SDN est très intéressant dans le cas de l'Urbanloop. En effet il permettrait de pouvoir gérer dynamiquement le réseau tout en étant facilement maintenable à distance.

III. EXTENSION DU SIMULATEUR D'URBANLOOP

A. introduction

Nous avons à notre disposition une simulation informatique issue d'un Projet Industriel 2018-2019 mené par Mlle ASSELIN-BOULLÉ, M CHOCOT et M HENRY et encadré par M CHOLEZ. L'objectif de ce projet était de créer un

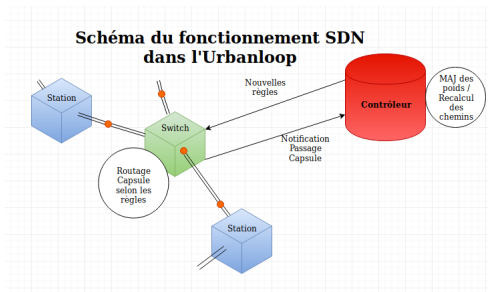


Fig. 1. Architecture SDN appliquée à Urbanloop

simulateur du réseau des capsules Urbanloop. Cependant cette simulation fonctionnait sur un réseau statique. En effet il n'y avait pas d'adaptation du routage des capsules dans le réseau dans le cas d'une congestion sur un axe du réseau ou même de la coupure d'une voie. Notre Projet Interdisciplinaire ou de Découverte de la Recherche avait pour objectif d'adapter cette simulation pour obtenir un réseau dynamique capable de s'ajuster en cas de coupure de voie ou de congestion.

Le principe de SDN ayant déjà été présenté dans la première partie nous n'y reviendrons donc pas en détail. Nous nous concentrerons sur l'implémentation de ce concept dans le cas de la simulation.

1) *Les règles*: Nous avons créé un objet *Rule* afin de pouvoir définir des règles. Ces règles sont définies par plusieurs attributs.

- L'identifiant de la boucle pour lequel elle est valable.
- L'identifiant du switch pour lequel elle est valable.
- La priorité de la capsule pour que la règle soit sélectionnée.
- Le fait que la capsule soit vide ou non.
- Un attribut change défini ci dessous.

Ces règles permettent d'affiner le routage des capsules afin de les acheminer au bon endroit. Finalement les règles possèdent également un attribut "change" qui indique si la capsule doit changer de boucle ou pas pour continuer son trajet, autrement dit l'attribut "change" permet d'orienter la capsule vers sa prochaine destination. Cet attribut est donc l'instruction que doit effectuer le switch si la règle correspond à la situation.

Ces règles sont primordiales lorsqu'une capsule arrive sur un switch. Lorsque cet événement se produit, on regarde si les informations portées par la capsule (destination, priorité, présence ou non d'un passager) permettent d'identifier une règle à appliquer. Si une règle correspond avec les informations de la capsule, on peut alors indiquer à la capsule si elle doit changer de boucle ou non et donc lui permettre de continuer sur le bon chemin. Les règles sont examinées dans l'ordre chronologique où elles ont été créées.

B. Calcul des itinéraires

Le calcul des itinéraires est réalisé par un algorithme de Dijkstra classique. Nous avons en effet estimé après avoir étudié l'état de l'art (voir annexe VII-B) que c'était la meilleure solution pour calculer les trajets les plus courts entre deux emplacements. Le calcul des itinéraires se fait à partir d'un graphe dont le poids des arcs est la durée de parcours du segment reliant les deux emplacements représentés par les deux noeuds. Nous avons représenté le graphe sous la forme d'une matrice d'adjacence où l'absence de chemin entre deux noeuds donne un poids infini. Dans le cas où un noeud ne serait pas atteignable, nous arrêtons la capsule au noeud précédent.

C. Congestion

Pour gérer la congestion nous avons mis en place un système équivalent au système de gestion des congestions de TCP Vegas. Il s'agit de mettre à jour le poids d'un segment (durée de parcours du segment) lorsqu'il a été parcouru par une capsule. On le met à jour en suivant la formule :

$$\text{Nouveau_poids} = (1 - 0.125) * \text{ancien_poids} + 0.125 * \text{temps_parcours} \quad (1)$$

Cette formule permet ainsi de faire augmenter progressivement le temps de parcours estimé du segment lorsqu'une congestion se crée (Étant donné que la durée de parcours constaté du segment sera alors plus grande que le temps de parcours estimé du segment). Parallèlement si le temps de parcours constaté se réduit et se trouve être inférieur au temps estimé le nouveau temps estimé sera également plus petit.

Nous déclarons une congestion lorsque le temps constaté est trois fois supérieur au temps estimé. On met alors à jour les tables de règles pour dérouter une partie des capsules vers une voie plus rapide. Lorsque la congestion est terminée (si on passe à un temps constaté moins de deux fois supérieur au temps estimé) on remet alors à jour les règles. Pour le moment ces rapports sont estimés arbitrairement et devront donc être vérifiés expérimentalement et si besoin est réajuster plus rigoureusement.

IV. LIMITES DU SIMULATEUR

Une limitation de la simulation qui nous était fournie était l'absence de "collisions" entre les capsules. En effet deux capsules pouvaient se chevaucher, aucune congestion ne pouvait donc se créer. Pour régler ce problème, nous avons empêché qu'une capsule puisse sortir d'une station ou d'un stockage si une autre capsule est en train d'arriver sur la station (stockage) ou vient d'en partir. Les capsules se déplaçant à la même vitesse dans le simulateur, elles ne peuvent pas se chevaucher. Cela repousse seulement l'entrée de la capsule sur le réseau. Le véritable problème était donc l'entrée d'une capsule déjà dans le réseau, sur une autre boucle. Pour régler cela, nous avons considéré comme solution l'arrêt de la capsule si l'entrée sur la boucle ne peut pas se faire (par exemple si une autre capsule est en train de passer). Cette manière de faire est certes assez peu réaliste mais reste néanmoins fonctionnelle et nous

aurait permis d'obtenir de la congestion. En effet les temps de franchissement des sections aurait évidemment augmenté si les capsules restaient à l'arrêt le temps de s'insérer. Nous nous sommes cependant heurté à la conception même du simulateur qui nous était fourni. En effet les capsules n'ont pas de position intrinsèque. Les positions des capsules ne sont pas calculées dynamiquement mais sur le temps de trajet estimé. En pratique, pour calculer la position d'une capsule sur un segment, le calcul suivant est effectué :

$$(Temps_actuel - Temps_dpart) * vitesse_capsule \quad (2)$$

Nous avons dans un premier temps pensé à passer la vitesse à zéro. Cependant non seulement cela remettait la position de la capsule au début du segment mais en plus dès lors que l'on remettait la vitesse normale de la capsule, le recalcul de sa position ne prenait pas en compte le fait que la capsule avait été arrêtée mais recalculait comme si la vitesse avait toujours été constante (ce qui est normal étant donné que le temps de départ du segment ne varie pas). Il en résultait une "téléportation" de la capsule dès lors que la position de celle-ci était recalculée.

Nous avons donc pensé à utiliser un attribut "real_time" plutôt que d'utiliser le temps de départ du segment. L'objectif était de prendre en compte le temps passé à l'arrêt par la capsule afin d'obtenir de meilleurs résultats. Il s'agissait de ne pas incrémenter l'attribut lorsque la capsule est à l'arrêt. Cependant lorsque le temps est mis à jour le problème précédent se représente.

Nous avons alors pensé à créer un attribut déterminant si la capsule était arrêtée ou pas. Si elle est arrêtée, on met à jour l'attribut. Lorsque celui-ci est vrai, on ne met pas à jour la capsule. Après avoir eu des problèmes de disparition de la capsule en question, nous avons finalement des problèmes similaires au problème précédent.

Enfin, le principal obstacle au calcul des "collisions" dépend donc de la conception même du simulateur. Le simulateur a été prévu pour la simulation d'un réseau statique. La transition vers un simulateur d'un réseau dynamique devra se faire par une refonte de nombreux points du simulateur actuel, en particulier passer d'une gestion prévisionnelle de la position des capsules à une gestion dynamique.

L'ajout de "collisions" entre les capsules aurait permis de créer de la congestion. La mise en place de la congestion étant primordiale pour tester le bien fondé de notre amélioration du système de routage existant, nous n'avons pu obtenir de résultats sur notre gestion de la congestion.

V. ÉVALUATION DES RÉSULTATS

A. Préambule

Dans cette section nous traiterons de la comparaison entre les résultats obtenus avec l'implémentation précédente du simulateur et la nouvelle implémentation du simulateur incluant l'architecture SDN. Nous n'avons cependant pas pu obtenir des résultats pour la congestion. En effet comme

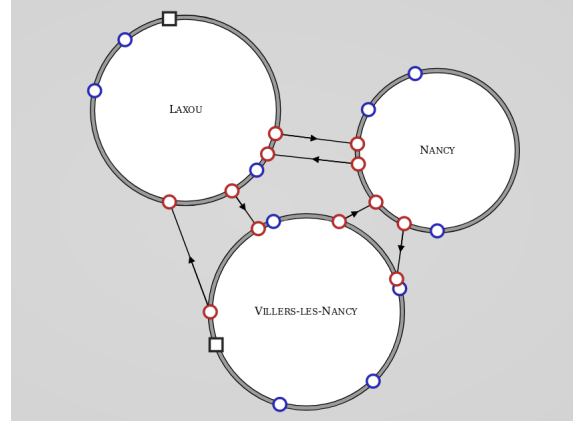


Fig. 2. Topologie du réseau Urbanloop utilisé pour les tests

précisé précédemment la conception du simulateur a posé problème.

Les simulations qui vont suivre ont été réalisées sur un même réseau (voir figure 2) comportant trois boucles reliées deux à deux par un switch entrant et un switch sortant. Le nombre de voyageurs moyen et la destination de ces voyageurs (et donc la durée du trajet) est aléatoire, cependant nous avons observé une plage de temps assez longue afin d'essayer de réduire au maximum cette influence. Nous avons en effet constaté que sur plusieurs simulations, la partie aléatoire influe sur les résultats finaux et nous avons essayé de limiter cela.

B. Fonctionnement hors coupure

Nous avons fait une première simulation dans le cas d'un fonctionnement normal sans coupure. Les données pour les deux simulateurs sont équivalentes ce qui est plutôt un bon résultat (voir figure 3). On constate un temps légèrement plus important pour le simulateur avec SDN, ce qui est dû à la refonte de l'approvisionnement en capsule. En effet la nouvelle version du simulateur réapprovisionne beaucoup plus souvent les stations. Il en résulte une augmentation du temps de trajet qui est parfois long car certaines stations sont loin des entrepôts. Cependant on peut constater que l'architecture SDN n'influe que très peu sur le temps moyens des capsules lorsqu'il n'y a pas de coupures.

Nous avons également récupéré le temps d'attente moyen des voyageurs (voir figure 4). Nous pouvons constater une amélioration avec notre implémentation du simulateur. En effet cela est dû à un approvisionnement des stations qui est plus efficace, les voyageurs ont donc besoin d'attendre moins longtemps avant qu'une capsule arrive pour les emmener à leur destination.

Cependant, on remarque que même si ce temps moyen semble se stabiliser au début, il se met à augmenter après un certain temps. Cela est dû à une erreur dans le simulateur qui a été révélée par ces résultats : certaines capsules disparaissent du réseau, le nombre global de capsule diminue durant la simulation, il est donc plus compliqué d'approvisionner les

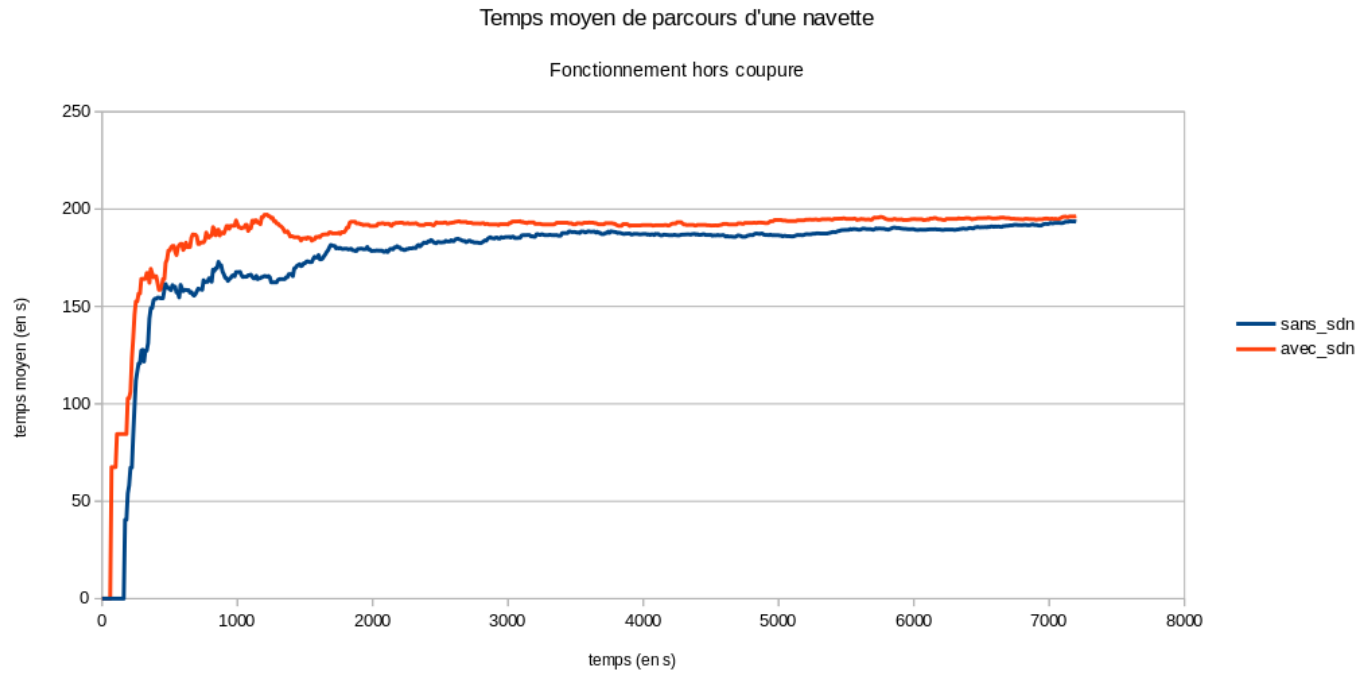


Fig. 3. Évolution du temps moyen de parcours d'une navette en condition nominale

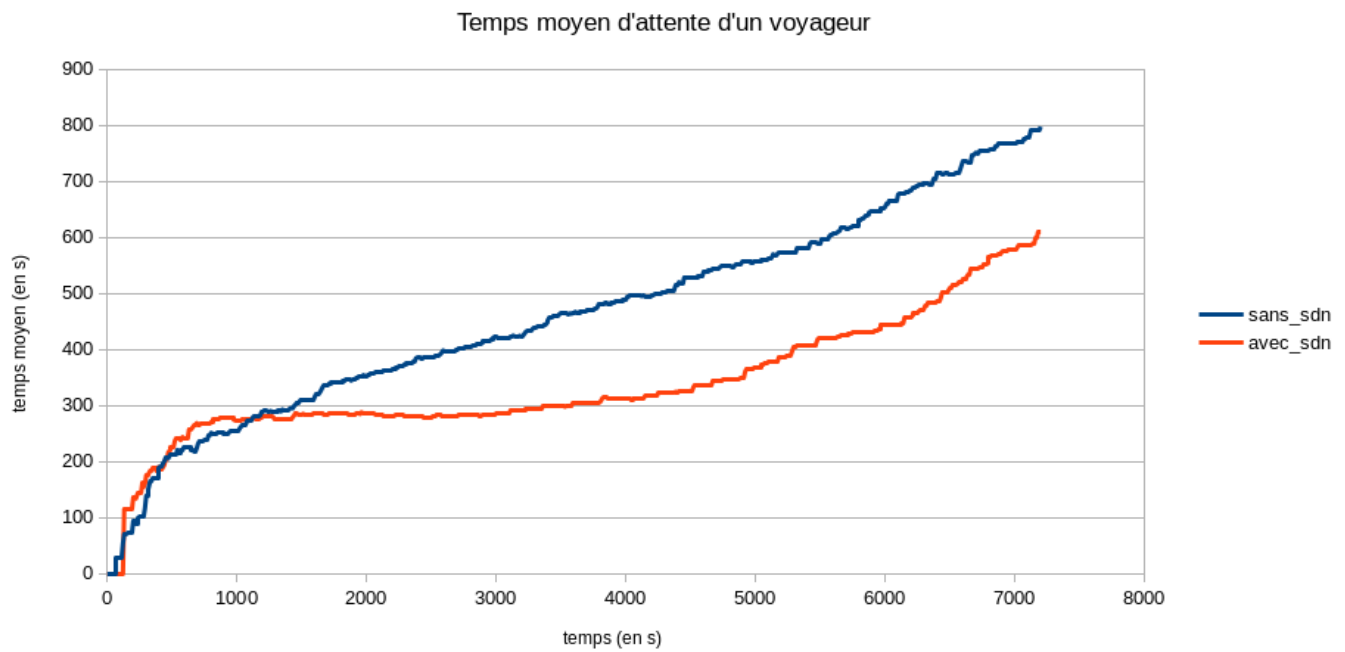


Fig. 4. Évolution du temps moyen d'attente d'un voyageur en condition nominale

stations comme voulu, ainsi le temps d'attente moyen des passagers augmente.

C. Fonctionnement avec coupure

Nous avons fait une première simulation dans le cas d'un fonctionnement normal avec coupure. Nous avons au bout d'un certain temps coupé une des voies reliant deux switches. Nous avons constaté que l'architecture SDN permet de répondre de manière effective à cette coupure. En effet, on peut observer que le temps moyen augmente ce qui est logique car la nouvelle route empruntée n'est plus la route optimale. Cependant malgré cette hausse du temps moyen, on constate que le système a répondu à cette anomalie. Au contraire, une coupure de voie n'était pas prévue dans le simulateur de base. Les routes n'étant pas recalculées une capsule cherchant à atteindre une station désormais inaccessible en suivant la route prévue s'arrête à la coupure. Elle augmente donc le temps moyen de trajet des capsules. Nous obtenons les courbes suivantes (voir figure 5).

VI. CONCLUSION

En conclusion, nous avons pu implémenter une simulation d'un réseau Urbanloop s'adaptant à différents problèmes pouvant intervenir lors du fonctionnement de l'Urbanloop. Les deux défauts pris en compte étant la coupure soudaine d'une voie et la congestion dans le réseau. Pour ce faire il a été décidé d'appliquer le paradigme SDN au simulateur déjà existant.

Un contrôleur a donc été créé : il connaît l'ensemble de la topologies du réseau ainsi que l'emplacement de chaque élément du réseau. Ainsi il peut créer un ensemble de règles qui seront suivies par les switches du réseau afin de router les capsules.

Ce contrôleur en tant que tel n'apporte pas d'amélioration au simulateur, cependant il permet par la suite de pouvoir adapter le routage à l'état du réseau.

La gestion d'une coupure soudaine est très concluante : les capsules s'adaptent à la nouvelle topologie du réseau ce qui n'était pas le cas avec l'ancienne simulation. Ainsi lors de la coupure d'une voie, toutes les capsules passant par cette voie ne pouvaient plus circuler, une grande partie du réseau n'était donc plus accessible. Dans notre implémentation, toutes les parties du réseau toujours accessibles continuent d'être alimentées. Ainsi nous obtenons un réseau s'adaptant dynamiquement à la coupure d'une voie ce qui n'était pas le cas. Nos tests ont montré que notre solution est efficace : le temps moyen des trajets des capsules ne fait que peu augmenter lors d'une coupure, tandis qu'avec l'ancien simulateur le temps moyen tend vers l'infini étant donné que les chemins ne sont pas recalculés.

Le gestion d'une congestion est basée sur le fonctionnement de TCP Vegas : si on atteint une valeur de seuil, on sait qu'on est en congestion, on peut alors recalculer les routes selon

les nouveaux poids du graphe recalculés dynamiquement à partir du temps de parcours échantillonné. Ainsi nous avons obtenu un réseau pouvant adapter le routage des capsules à la circulation du réseau. Cependant, nous n'avons pu réellement tester ce qui a été mis en place car le simulateur disponible n'était pas pensé pour pouvoir créer de la congestion, les capsules se superposant et allant donc à une vitesse linéaire.

Ainsi nous avons obtenu une nette amélioration du simulateur existant grâce à l'application du paradigme SDN, cependant par manque de temps, toutes les fonctions implémentées n'ont pu être testées.

REMERCIEMENTS

Nous souhaiterions dans un premier temps remercier M Thibault Cholez pour son aide et pour le temps qu'il a consacré pour avec nous échanger à propos du projet. Ces réunions ont été précieuses pour orienter nos réflexions. Nous souhaiterions également remercier Mlle Asselin-Boullé et MM Chocot et Henry pour leur simulateur qui a servi de base à notre travail.

Nous remercions M Mangin pour sa coordination du projet Urbanloop auquel nous avons eu la chance de prendre part. Nous remercions finalement Télécom Nancy de nous avoir donné l'opportunité de collaborer au projet Urbanloop.

REFERENCES

- [1] Diego Kreutz, Fernando M. V. Ramos, Paulo Verissimo, Christian Esteve Rothenberg, Siamak Azodolmolky, and Steve Uhlig. Software-defined networking: A comprehensive survey. Technical report, IEEE, 2014.
- [2] Stéphane Bortzmeyer. Rfc 7098: Using the ipv6 flow label for load balancing in server farms, jan 2014.
- [3] Frédéric Launay. Principe du sdn dans une architecture réseau classique, jan 2018.
- [4] Saurabh Kumar Agarwal. Understanding openflow. Technical report, Bharati Vidyapeeth College of Engineering, jun 2014.
- [5] sdxCentral. What is openflow? definition and how it relates to sdn.
- [6] Maxence Tury. Les risques d'openflow et du sdn. Technical report, ANSSI, 2014.
- [7] Margaret Rouse. Nfv (network functions virtualization). *LeMagIt*, jul 2015.
- [8] Larousse. Définition recherche opérationnelle, 1966.

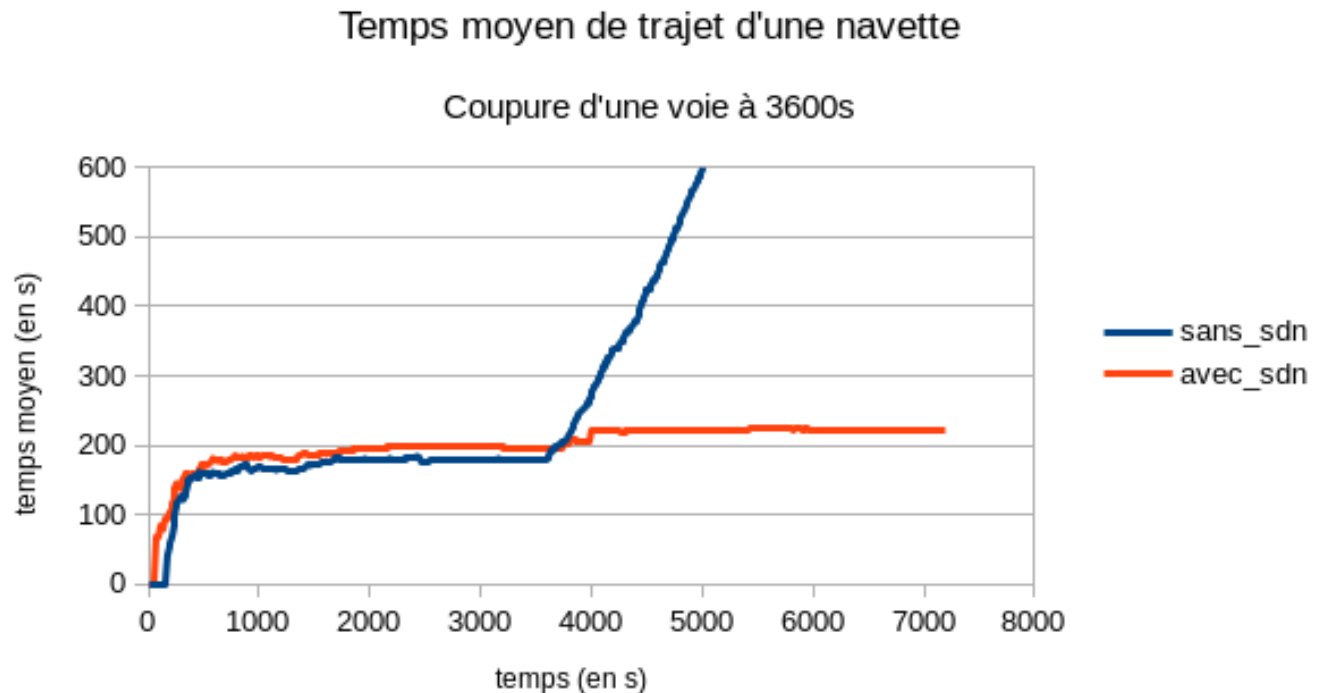


Fig. 5. Évolution du temps moyen de parcours d'une navette après un incident à $t=3600s$

VII. ANNEXES

A. Compléments sur SDN

1) *Network hypervisors*: L'objectif d'un hyperviseur est de permettre à plusieurs machines virtuelles de partager le même matériel physique. Cet hyperviseur, installé sur la machine contrôle l'accès au matériel et partage les ressources physiques entre toutes les machines virtuelles. Un des problèmes actuel est le fait qu'il est difficile d'avoir plusieurs topologies de travail virtuelles sur une seule machine physique et il en va de même pour les champs d'adressage. Par exemple, une machine physique configurée pour l'IPv4 ne peut pas traiter de l'IPv6.[1][7]

Ainsi pour arriver à rendre virtuel le contrôle du réseau, il faut standardiser les charges de travail et les équipements physiques.

Flowvisor est un outil de virtualisation de réseau. Son objectif est de permettre à différents réseaux logiques de partager la même infrastructure. Il s'agit de découper le plan de données pour le partager entre plusieurs contrôleurs, un contrôleur gère alors uniquement sa partie. Flowvisor s'occupe de la répartition des contrôleurs pour les flow tables d'un équipement.[1]

En conclusion la virtualisation des réseaux peut être un énorme atout pour le principe de SDN mais les solutions actuelles ne sont pas satisfaisantes.

2) *Alternatives à l'OpenFlow SDN*: Il existe des alternatives au système OpenFlow. Par exemple l'Overlay SDN qui utilise la technologie de tunneling : on ajoute une couche au réseau qui fait tunnel pour permettre au programmeur d'interagir avec le plan de contrôle qu'on n'a donc pas eu besoin de changer. C'est à dire qu'on crée un réseau virtuel indépendant du réseau physique. Ce réseau virtuel s'appuie sur des hyperviseur, ainsi les routeurs sont virtualisés. Le matériel physique ne sert alors plus que de support et de connecteur.[1][3]

Plusieurs essais ont été effectués dans le sens de l'Overlay SDN (approche par couches), comme par exemple mettre le management plane au même niveau que le control plane. Le management plane peut alors être considéré comme une plate-forme de contrôle intégrant les protocoles de réseau traditionnels.[1][3]

3) *L'initiative Network Functions Virtualization*: Cette initiative s'inscrit dans la volonté de développement de SDN. Il s'agit de virtualiser des services réseau déployés par un matériel dédié et propriétaire. L'objectif est d'apporter une plus grande flexibilité et une plus grande programmabilité. En effet virtualiser des services sur des machines virtuelles, permettrait de réduire grandement le nombre de matériel dédié nécessaire au fonctionnement du réseau. De plus la modification d'un service serait alors une modification software et ne nécessiterait plus de modification du hardware.[1]

B. Algorithme de recherche opérationnelle - État de l'Art

1) Introduction:

a) *Définition* : La recherche opérationnelle peut être définie comme l'ensemble des méthodes et techniques rationnelles orientées vers la recherche du meilleur choix dans la façon d'opérer en vue d'aboutir au résultat visé ou au meilleur résultat possible.[8]

b) *Cadre* : Nous nous plaçons dans le cadre d'une recherche d'un chemin optimal dans un graphe pondéré et orienté. En effet nous considérerons le réseau comme un graphe. Les noeuds sont les switches et les stations. Les arrêtes sont les tronçons entre ces objets. Les poids des arrêtes sont définis en fonction du temps qu'il faut pour parcourir un tronçon. Plus un tronçon sera rapide à parcourir plus son poids sera faible. Ainsi un tronçon très encombré sera représenté par une arrête possédant un poids important. Le graphe étudié est orienté car les capsules ne peuvent emprunter les tronçons que dans un seul sens.

2) Algorithmes:

a) *Préambule* : Nous allons voir différentes méthodes de recherche opérationnelles :

- Algorithme de Dijkstra
- Algorithme A*
- Algorithme de Floyd-Warshall
- Algorithme de Bellman-Ford

b) *Algorithme de Dijkstra* : L'algorithme de Dijkstra permet de trouver le plus court chemin dans un graphe valué pondéré à valeurs positives. Il est basé sur une recherche locale du plus court chemin. Il s'agit d'un algorithme glouton.

L'algorithme de Dijkstra est un algorithme facilement implémentable qui donne plutôt de bons résultats. Un avantage dans notre cas est qu'il est celui déjà implémenté dans la simulation actuellement. Cependant l'algorithme de Dijkstra n'utilise pas toutes les informations que nous avons à notre disposition. En effet il recherche "à l'aveugle" la destination sans tenir compte que nous avons une carte du réseau.

c) *Algorithme de Bellman-Ford* : L'algorithme de Bellman-Ford permet de trouver le plus court chemin dans un graphe valué pondéré à valeurs positives ou négatives.

L'algorithme de Bellman-Ford est moins efficace que Dijkstra en terme de temps d'exécution. Son principal avantage serait de pouvoir utiliser des arcs à poids négatifs ce qui est inutile dans notre cas.

d) *Algorithme A** : L'algorithme A* se veut une amélioration de l'algorithme de Dijkstra. A* à l'avantage sur Dijkstra de ne pas explorer les voies les moins prometteuses.

L'algorithme A* est plus efficace que l'algorithme de Dijkstra en terme de temps d'exécution. Il prend en compte

une évaluation de la distance par rapport à la destination. On peut lui fournir cette information étant donné qu'on sait où se trouve la destination dans le réseau. Cependant A* ne donne pas forcément la route la plus rapide. En effet en fonction de la configuration du réseau, un chemin qui ferait un grand détour ne serait pas forcément détecté même si il est plus rapide du au poids des arrêtes traversées. Dans notre cas au vu de la vitesse de déplacement dans les tubes, on peut difficilement être sûr que le temps de trajet en passant par une boucle qui ne va pas dans le sens de la destination sera plus long qu'un chemin qui va droit au but si il est congestionné. A* révèle véritablement son potentiel dans des graphes très importants ce qui n'est pas forcément notre cas.

e) *Algorithme de Floyd-Warshall* : L'algorithme de Floyd-Warshall est un algorithme pour déterminer les distances des plus courts chemins entre toutes les paires de sommets dans un graphe orienté et pondéré.

L'algorithme de Floyd-Warshall peut nous être utile selon la méthode que nous voulons employer. Si nous optons pour une approche "prévisionnelle" il peut être l'algorithme recherché. Il est inutile dans le cas contraire.

f) *Bilan* : On peut éliminer l'algorithme de Bellman-Ford qui n'a pas vraiment d'avantage par rapport aux deux autres. Entre l'algorithme de Dijkstra et l'algorithme A*, il est difficile de trancher. Il n'est en effet pas sûr que l'on veuille préférer la vitesse de calcul du plus court chemin à son exactitude.

C. Routage - État de l'Art

a) *Qu'est ce que le routage IP ?* : Le routage consiste à définir le chemin que vont emprunter les données à transmettre, c'est à dire par quel routeur les données vont transiter afin de rejoindre le réseau du destinataire depuis le réseau de l'émetteur.

Le routage IP sera défini de proche en proche. On dit que le routage IP est décentralisé car il n'y a pas de vision globale de la route des données, chaque routeur calcule quel sera le prochain routeur.

Le routage IP dépend donc du matériel pour calculer le routage. Chaque appareil possède une table de routage. Cette table contient des adresses réseaux ainsi que leur masque et surtout comment y accéder (soit l'adresse du prochain réseau, soit l'adresse, soit le nom de l'interface si le réseau est connecté à l'appareil).

Le protocole IP fonctionne de la manière suivante : lorsqu'un paquet de données arrive dans le routeur, on recherche une correspondance entre la destination du paquet et un réseau enregistré dans la table de routage. Si on trouve une correspondance on fait suivre le paquet selon les informations définies par la table, sinon on l'achemine au routeur par défaut (si un routeur par défaut a été défini).

b) *Les protocoles de routage interne* : Les deux protocoles que nous allons présenter par la suite sont des protocoles de routage interne ou IGP (Interior Gateway Protocol). Les protocoles internes permettent la gestion des routeurs à l'intérieur d'un système autonome c'est à dire un ensemble de réseaux gérés par un administrateur commun. Une de leur principale caractéristique (sur les réseaux à diffusion) est de trouver automatiquement les autres routeurs et d'obtenir la topologie du réseau pour déterminer la route afin de transmettre les données. Ces deux protocoles ne s'occupent pas de la communication inter-domaines pour laquelle on utilise des protocoles de type Exterior Gateway Protocol (BGP par exemple). Il existe d'autres protocoles de routage interne.

c) *Le Routing Information Protocol* : Le protocole RIP est un protocole de routage dit dynamique c'est à dire qu'à la place d'un codeur qui programme les adresses des réseaux à la main au niveau de chaque routeur, le protocole RIP va permettre au routeur de calculer les adresses des réseaux qui lui sont proches et ainsi de s'adapter en cas d'un incident (panne, coupure, changement, etc.).

Le protocole RIP est adapté pour traiter de petits réseaux (diamètre ; à 15 routeurs). Un des problèmes du protocole RIP est qu'il se base sur une métrique statique, c'est à dire que l'on considère que le coût pour atteindre un réseau voisin reste constant. Le protocole RIP a un temps de convergence de quelques minutes.

L'algorithme du protocole RIP est basé sur celui de Bellemann-Ford (algorithme de parcours de graphe).

L'algorithme de routage de RIP est un protocole de routage à vecteurs de distance. On dit de ces protocoles qu'ils sont itératifs (ils continuent de fonctionner tant qu'il y a des informations à échanger), asynchrones (les routeurs sont indépendants) et distribués (aucun routeur ne connaît la topologie complète du réseau). L'échange d'informations ne se fait qu'entre routeurs voisins dans le réseau. Le routage à vecteurs de distance fonctionne de la manière suivante : lorsqu'un routeur reçoit une nouvelle table de routage, pour chaque adresse il la rentre dans la table si elle n'y était pas. Si elle y était, il met à jour le coût de la route dans le cas où la nouvelle route (coût présent dans la table émise + coût du chemin depuis l'émetteur jusqu'au routeur) serait plus courte que celle déjà présente dans la table. Lorsque la table a été modifiée, elle est envoyée aux ports voisins. Et cela jusqu'à ce que l'algorithme converge. Le coût est appelé métrique.

d) *L'Open Shortest Path First Protocol*: L'OSPF a été conçu pour améliorer le protocole RIP. Le temps de convergence est meilleur, on peut utiliser des réseaux plus importants (plus de 16 routeurs), la métrique n'est plus statique mais dynamique (elle s'adapte aux débits).

OSPF utilise l'algorithme de Dijkstra (algorithme de recherche du plus court chemin dans un graphe) et l'algorithme

d'état de lien (Link State).

L'algorithme de routage de OSPF est dit Link State. Les routeurs émettent dès lors qu'un état de lien évolue. Chaque routeur envoie la liste de ses voisins à chaque autre routeur du réseau. Finalement tous les routeurs ont accès à la topologie du réseau et aux coûts des liens entre routeurs. Ces informations sont propagées grâce à des Link State Packet (LSP). Attention, il est important que chaque routeur ait connaissance de ses voisins et de l'état du lien qui les unit.

OSPF possède une particularité, un routeur est désigné (Designated Routeur). Les routeurs n'envoient la liste de leurs voisins qu'au DR. Ce dernier se charge d'envoyer la topologie complète du réseau aux autres routeurs, ce qui permet d'améliorer la convergence. Le DR est le routeur qui possède la plus grande priorité (ou la plus grande adresse IP s'il y a égalité). Un Backup DR (BDR) est aussi désigné.

Chaque routeur demande au DR un résumé de sa base de données topologique. Ce dernier leur renvoie un Link State Advertisement (LSA). Grâce au LSA les routeurs savent si ils sont à jour. Dans le cas contraire, ils envoient au DR des Link State Request (LSR). Le DR renvoie un Link State Update (LSU) qui permet au routeur de se mettre à jour. Le DR renvoie alors un Link State Update (LSU) qui permet au routeur de se mettre à jour. Une fois à jour, il peut calculer les routes et les transmettre au DR si elles sont meilleures. Pour déterminer le coût des routes OSPF utilise l'algorithme de Dijkstra.

C'est un routage dynamique. Ainsi si un routeur détecte une modification de ses liens, il transmet la modification au DR qui se charge de la transmettre au réseau.

Il existe d'autre méthode de routage comme par exemple LDP (Label Distribution Protocol qui repose sur un système de label). Un label étant un en-tête MPLS. Une autre méthode est BGP qui s'utilise dans le contexte d'une communication inter-domaine.